

Análisis de algoritmos

Escuela Politécnica Superior, UAM, 2026–2027

Práctica n.º 1

Fecha de entrega de la práctica

- Grupos del miércoles: 20 de octubre de 2026.
- Grupos del jueves: 21 de octubre de 2026.

Los códigos fuente y la documentación de las prácticas de la asignatura de Análisis de algoritmos se entregarán en formato electrónico en la fecha indicada, a través de la plataforma Moodle (<https://moodle.uam.es/>).

La utilización de herramientas de IA está permitida como ayuda en las actividades indicadas, pero **en el examen no se podrá utilizar**. Las respuestas generadas por estas herramientas no se considerarán evidencia de corrección. Cada estudiante será responsable de verificar mediante razonamiento, experimentación, código o fuentes bibliográficas todas las afirmaciones que incorpore a la memoria. Utilizad siempre el enfoque «*Think first, ChatGPT later*».

INTRODUCCIÓN

Esta primera práctica servirá de base para las siguientes. Se crearán los ficheros fuente `permutations.c`, `sorting.c` y `times.c`, en los que se implementarán las distintas funciones en C para su posterior reutilización. Los correspondientes ficheros de cabecera, `permutations.h`, `sorting.h` y `times.h`, incluirán los prototipos de estas funciones y las macrodefiniciones de uso general, como las siguientes:

```
#define ERR    -1
#define OK     (! (ERR))
```

Será obligatorio ajustarse a las plantillas proporcionadas: los prototipos de las funciones no deben modificarse, salvo indicación del profesor de prácticas.

Recomendaciones para la implementación de las funciones:

- Comprobad los argumentos de entrada y devolved el valor de error especificado cuando no sean válidos. Podéis utilizar `assert` para comprobar invariantes internos durante la depuración, pero no como sustituto de la gestión de errores, ya que puede desactivarse y, si falla, aborta el programa. Evitad que las funciones dependan de variables globales.
- Liberad la memoria y cerrad los ficheros cuya gestión corresponda a la función y que ya no se necesiten, también en las salidas por error. No liberéis la memoria que debáis devolver al llamador ni los recursos que este deba seguir utilizando.
- Comprobad que los ficheros se pueden abrir y que las operaciones de escritura y cierre se completan correctamente. En caso de error, proceded como se indica en el punto b y notificad el fallo mediante el valor de retorno correspondiente.
- Comprobad que cada reserva de memoria se ha realizado correctamente. Si falla, proceded como se indica en el punto b antes de salir de la función o del programa.
- Probad los programas con casos extremos y entradas erróneas (por ejemplo, tablas muy grandes o argumentos no válidos) para verificar su correcto funcionamiento y la gestión adecuada de los errores.
- Las prácticas se corregirán en un sistema Unix o Linux. Los programas deberán ajustarse a C99, de acuerdo con las opciones del Makefile proporcionado, y utilizar funciones de la biblioteca estándar para favorecer su portabilidad.**

En la plataforma Moodle está disponible el fichero `practice1.zip`, que contiene las plantillas en C con los prototipos de las funciones requeridas en esta práctica, así como programas de prueba para cada apartado. Estos programas son un punto de partida: debéis comprobar qué función invocan y completar las pruebas de casos extremos y errores.

En primer lugar, se implementarán en el fichero `permutations.c` funciones en C para generar permutaciones aleatorias de un número dado de elementos. Estas permutaciones servirán como entradas de los algoritmos de ordenación. A continuación, se implementarán en el fichero `sorting.c` los algoritmos de ordenación propuestos en la práctica. Por último, se desarrollarán en el fichero `times.c` funciones para medir sus tiempos de ejecución utilizando las permutaciones generadas.

PRIMER BLOQUE

Generación de permutaciones

1. La función `rand`, declarada en `stdlib.h`, genera enteros pseudoaleatorios entre 0 y `RAND_MAX`, ambos incluidos. La biblioteca de C no garantiza la calidad estadística de la secuencia; para el análisis de esta práctica se supondrá que sus valores son equiprobables y que las llamadas sucesivas son independientes. Utilizad esta función para implementar en `permutations.c` la función `int random_num(int inf, int sup)`, que debe generar un entero equiprobable entre `inf` y `sup`, ambos incluidos, bajo esas hipótesis. Para esta práctica se trabajará con $0 \leq \text{inf} \leq \text{sup}$ y con un tamaño de intervalo representable como `int` que no supere `RAND_MAX + 1`. Devolved `ERR` si los argumentos no cumplen estas condiciones y evitad desbordamientos en los cálculos intermedios. Inicializad la semilla con `srand` una sola vez en el programa principal, no en cada llamada a `random_num`; los programas de prueba ya incluyen esa inicialización. Utilizad el fichero `exercise1.c` para comprobar el correcto funcionamiento de la función que acabáis de implementar. Elaborad un histograma con una barra por cada entero del intervalo y comparad las frecuencias observadas con las esperadas para una distribución uniforme. El histograma permite detectar desviaciones, pero no demuestra la equiprobabilidad. Podéis usar IA como ayuda para generar una gráfica con GNUPlot a partir de vuestros datos; comprobad que los representa fielmente e indicad el intervalo y el número de muestras utilizados.
2. El siguiente pseudocódigo corresponde al algoritmo de mezcla de Fisher–Yates para generar permutaciones aleatorias:

```
para i de 1 a N:
    perm[i] = i;
```

```
para i de 1 a N:
    intercambiar perm[i] con perm[random_num(i, N)];
```

donde `random_num` es la función del ejercicio anterior. Implementad en el fichero `permutations.c` una función C `int *generate_perm(int N)` para dicho algoritmo. La función reservará memoria para la permutación y devolverá un puntero a ella, o `NULL` si $N \leq 0$, si falla la reserva de memoria o si no se puede generar un índice válido. El llamador será responsable de liberar la permutación cuando deje de utilizarla. Podéis utilizar una IA para traducir el pseudocódigo a C. Tened en cuenta que los índices de un array de N elementos van de 0 a $N - 1$, mientras que los valores de la permutación son los enteros del 1 al N . Revisad y mejorad el código generado y comprobad que los límites de los bucles y de las llamadas a `random_num` se adaptan correctamente a esta indexación. Utilizad el fichero `exercise2.c` para comprobar el correcto funcionamiento de la función que acabáis de implementar.

SEGUNDO BLOQUE

3. Implementad en el fichero `permutations.c` la función `int** generate_permutations(int n_perms, int N)`, que generará, mediante `generate_perm`, `n_perms` permutaciones de N elementos cada una. Cada permutación se generará de forma independiente, por lo que pueden repetirse. Ambos argumentos deben ser positivos. En caso de error, liberad las reservas parciales y devolved `NULL`. El llamador deberá liberar cada permutación y después el array de punteros. Utilizad el fichero `exercise3.c` para comprobar el correcto funcionamiento de la función que acabáis de implementar.

Algoritmos de ordenación

En esta sección se implementará el algoritmo de ordenación por inserción *InsertSort* y se contará el número de ejecuciones de su **operación básica** (OB). En los apartados siguientes se medirán el tiempo medio de ejecución y los números medio, mínimo y máximo de ejecuciones de la OB sobre muestras de permutaciones de distintos tamaños. Los resultados se compararán con el análisis teórico; los extremos observados en una muestra no tienen por qué coincidir con los casos mejor y peor teóricos.

4. Implementad en `sorting.c` la función `int InsertSort(int *array, int ip, int iu)` para el método de ordenación por inserción visto en teoría. La función ordenará de menor a mayor el segmento de `array` comprendido entre los índices `ip` e `iu`, ambos incluidos, sin modificar los elementos exteriores al segmento. El puntero debe ser válido y los índices deben cumplir $0 \leq ip \leq iu$; el llamador debe garantizar que el array contiene esas posiciones. La función devolverá `ERR` en caso de error o el número de ejecuciones de la OB en caso de éxito. Indicad qué operación contáis y contad todas sus evaluaciones, incluidas las que produzcan un resultado falso. Utilizad tamaños para los que el recuento sea representable como `int`.

Utilizad `exercise4.c` para comprobar la función. La plantilla incluida invoca `BubbleSort`: sustituid esa llamada por `InsertSort` y actualizad el mensaje de error correspondiente para este apartado. Comprobad también segmentos que no empiecen en el índice 0 y tablas de un solo elemento.

TERCER BLOQUE

Tiempos de ejecución

5. Utilizad en `times.h` la siguiente estructura, ya declarada en la plantilla, para almacenar las estadísticas de un algoritmo sobre un conjunto de permutaciones:

```
typedef struct time_aa {
    int N;                /* permutation size */
    int n_elems;          /* number of permutations averaged */
    double time;          /* average CPU time in seconds */
    double average_ob;    /* average number of OB executions */
    int min_ob;           /* minimum number of OB executions */
    int max_ob;           /* maximum number of OB executions */
} TIME_AA, *PTIME_AA;
```

Implementad en el fichero `times.c` la función:

```
short average_sorting_time(pfunc_sort method, int n_perms, int N, PTIME_AA ptime)
```

donde `pfunc_sort` es un tipo de puntero a una función de ordenación, definido como:

```
typedef int (*pfunc_sort)(int*, int, int);
```

Esta definición de tipo ya está incluida en `sorting.h`. El argumento `method` indica la función de ordenación, `n_perms` es el número de permutaciones que se generarán y ordenarán, `N` es el tamaño de cada permutación y `ptime` apunta a una estructura `TIME_AA`. Los punteros deben ser válidos y ambos enteros deben ser positivos. En caso de éxito, la estructura contendrá:

- el número de permutaciones promediadas en el campo `n_elems`,
- el tamaño de las permutaciones en el campo `N`,
- el tiempo medio de CPU por permutación (en segundos) en el campo `time`,
- el número promedio de veces que se ejecutó la OB en el campo `average_ob`,
- el número mínimo de ejecuciones de la OB en el campo `min_ob` y
- el número máximo de veces que se ejecutó la OB en el campo `max_ob`.

Estas estadísticas corresponderán a `method`, aplicado con índices 0 y $N - 1$ a las permutaciones generadas mediante `generate_permutations`. Generad las entradas antes de iniciar la medición y exclud de ella la generación, la reserva y liberación de memoria y la escritura de resultados. Medid con `clock`, declarada en `time.h`, el tiempo acumulado del lote de ordenaciones; convertidlo a segundos mediante `CLOCKS_PER_SEC` y divididlo entre `n_perms`, evitando la división entera. Comprobad el valor de error `(clock.t)-1`. El tiempo así obtenido es tiempo de CPU, no tiempo de pared. Calculad el promedio de OB sin desbordar la suma y liberad las permutaciones al terminar, también si se produce un error.

La función `average_sorting_time` devuelve `ERR` en caso de error y `OK` en el caso de que las tablas se ordenen correctamente.

Implementad además la función:

```
short generate_sorting_times(pfunc_sort method, char* file, int num_min, int num_max,
                           int incr, int n_perms)
```

Esta función llamará a `average_sorting_time` para cada tamaño $N = \text{num_min} + k \cdot \text{incr}$, con $k = 0, 1, 2, \dots$, mientras $N \leq \text{num_max}$, y guardará las estadísticas en el fichero `file`. El tamaño `num_max` solo se incluirá si se alcanza con esos incrementos. Debe cumplirse $1 \leq \text{num_min} \leq \text{num_max}$, con `incr` y `n_perms` positivos, y los punteros deben ser válidos. Evitad desbordamientos al calcular los tamaños y las reservas. La función devolverá `ERR` en caso de error y `OK` en caso contrario.

Implementad la función `generate_sorting_times` para que llame a:

```
short save_time_table(char* file, PTIME_AA time, int n_times)
```

Esta función escribirá una fila por tamaño y seis columnas, en este orden: `N`, `n_elems`, `time`, `average_ob`, `min_ob` y `max_ob`. El campo `n_elems` contendrá el valor de `n_perms` y `time` expresará el tiempo medio de CPU en segundos. El parámetro `time`, de tipo `PTIME_AA` (equivalente a `TIME_AA*`), apunta al primer elemento del array de estadísticas; `n_times` es su número de elementos y debe ser positivo. Comprobad los punteros y los errores de apertura, escritura y cierre. La función devolverá `ERR` si se produce un error y `OK` en caso contrario.

Utilizad `exercise5.c` para medir `InsertSort`. La plantilla pasa `BubbleSort` a `generate_sorting_times`: cambiad ese argumento por `InsertSort` para este apartado.

Comentario: Una medición puede dar cero si el intervalo medido es menor que la resolución efectiva del reloj. También puede mostrarse como cero por una división entera o por usar pocos decimales al escribir el resultado. Revisad estos aspectos antes de interpretar los datos. No introduzcáis retardos artificiales, pues no miden el coste de la ordenación.

Para obtener mediciones más fiables, aumentad el número de permutaciones del lote o su tamaño. Medid el lote completo, no suméis mediciones individuales que ya se hayan redondeado a cero. Si repetís una medición, utilizad nuevas entradas o copias sin ordenar, preparadas fuera del intervalo medido.

6. Implementad en `sorting.c` la función `int BubbleSort(int* array, int ip, int iu)` con el mismo significado de los argumentos, sentido de ordenación y valores de retorno que `InsertSort`. Indicad la variante de burbuja utilizada, en particular si incluye parada anticipada cuando no hay intercambios, y mantened esa variante en las pruebas y en el análisis teórico. Identificad la OB y comparad sus recuentos y tiempos medios de CPU con los de `InsertSort`. Adaptad `exercise4.c` y `exercise5.c` para probar y medir ambos métodos, guardando sus resultados en ficheros distintos. Para la comparación, utilizad copias de las mismas permutaciones sin ordenar y mantened las mismas condiciones de compilación y ejecución.

Cuestiones sobre la práctica

1. Considerad la implementación de `random_num` vista en clase, basada en aritmética entera y en el operador módulo. Para concretar el análisis, considerad la expresión `inf + rand() % (sup - inf + 1)` bajo las condiciones de entrada del apartado 1 y las hipótesis de uniformidad e independencia allí indicadas. Podéis consultar el capítulo 7 (*Random Numbers*) del libro *Numerical Recipes in C: The Art of Scientific Computing* o podéis utilizar una herramienta de IA para investigar si esta implementación garantiza la equiprobabilidad de todos los enteros del intervalo. Identificad el problema que puede aparecer y determinad en qué condiciones se produce. Verificad la explicación mediante un ejemplo reducido en el que podáis contar cuántos valores de origen se asignan a cada resultado. Proponed una alternativa que evite el problema sin recurrir a aritmética en coma flotante, describidla mediante pseudocódigo o código C y justificad por qué produce resultados equiprobables bajo las hipótesis anteriores. Comentad su coste respecto a la implementación propuesta. Si utilizáis IA, indicad la herramienta empleada, las consultas realizadas y qué afirmaciones de sus respuestas habéis comprobado o corregido; no basta con reproducir la respuesta obtenida.
2. Utilizad una herramienta de IA para obtener una propuesta de demostración de que el algoritmo de mezcla de Fisher–Yates, en la versión implementada en esta práctica, genera permutaciones equiprobables. Suponed que los N elementos son distintos y que, en cada iteración, la posición de intercambio se elige uniformemente entre la posición actual y la última, con independencia de las elecciones anteriores. Adaptad los índices de la demostración al código C: si la posición actual es i , la llamada será `random_num(i, N-1)`. Verificad cada paso del razonamiento y justificad que cualquier permutación final tiene probabilidad $1/N!$. Explicad por qué la garantía de equiprobabilidad puede dejar de cumplirse si `random_num` introduce un sesgo. Indicad la herramienta empleada, las consultas realizadas y los posibles errores o lagunas que hayáis detectado y corregido en su respuesta.

3. Justificad lo más formalmente que podáis la corrección del algoritmo **InsertSort** (o dicho de otra manera, por qué ordena correctamente). Utilizad una herramienta de IA para obtener una propuesta de demostración a partir de vuestro código y verificad cada paso de su razonamiento, comprobando que se aplica a vuestra implementación. Corregid los posibles errores o lagunas y presentad una justificación propia de la corrección y la terminación del algoritmo; no basta con comprobar que ordena algunos ejemplos. Indicad la herramienta empleada, las consultas realizadas y el resultado de vuestra verificación.
4. Utilizad una herramienta de IA para generar una explicación de los límites del bucle exterior de vuestra implementación de **InsertSort**. Comprobad si es válida la afirmación de que este bucle no actúa sobre el último elemento de la tabla; no la deis por cierta de antemano. Verificad la respuesta de la IA examinando los valores inicial y final del índice, la condición de continuación y los elementos procesados, y realizad una traza sobre una tabla pequeña. Si la afirmación o la explicación son incorrectas, corregidlas y justificad vuestra conclusión. Indicad la herramienta empleada, la consulta realizada y el resultado de la verificación.
5. ¿Cuál es la operación básica de **InsertSort**? Pedid a una herramienta de IA que proponga otras operaciones básicas que permitan analizar el coste de vuestra implementación. Comprobad en vuestro código si cada propuesta representa adecuadamente el coste del algoritmo. Indicad la herramienta empleada, la consulta realizada y los posibles errores que hayáis detectado y corregido en su respuesta.
6. Calculad el coste teórico, medido como el número de ejecuciones de la operación básica, en función del tamaño de entrada n , para los casos peor y mejor de **InsertSort** y **BubbleSort**. Denotad estos costes por $W_{IS}(n)$ y $B_{IS}(n)$ para **InsertSort**, y por $W_{BS}(n)$ y $B_{BS}(n)$ para **BubbleSort**. Utilizad la notación asintótica (O, Θ, o, Ω , etc.) cuando proceda. Utilizad también una herramienta de IA para obtener una propuesta de cálculo del coste medio $A_{IS}(n)$ y $A_{BS}(n)$, suponiendo que las $n!$ permutaciones de n elementos distintos son equiprobables. Facilitad a la IA vuestras implementaciones y especificad la operación básica que contáis en cada una. Verificad las hipótesis y cada paso de la derivación, y contrastad las expresiones obtenidas con los resultados experimentales de la práctica. Interpretad las diferencias entre los costes medios de ambos algoritmos y sus casos mejor y peor. No confundáis el valor esperado teórico con la media de una muestra finita. Indicad la herramienta empleada, las consultas realizadas y los errores que hayáis detectado y corregido en sus respuestas.
7. Utilizad una herramienta de IA para generar con GNUPlot, a partir de las tablas de tiempos obtenidas en la práctica, gráficas comparativas de **InsertSort** y **BubbleSort**. Representad los tiempos medios de CPU en función del tamaño de entrada de ambos algoritmos y, en gráficas separadas para cada algoritmo, los números medio, mínimo y máximo de ejecuciones de la operación básica. Incluid títulos, leyendas y etiquetas de los ejes con sus unidades, y utilizad escalas comunes para comparar ambos algoritmos. Comprobad que las gráficas corresponden a las seis columnas especificadas y que no se han alterado los datos. Los mínimos y máximos observados en la muestra no necesariamente coinciden con los casos mejor y peor teóricos. Interpretad las gráficas y justificad las similitudes y diferencias entre ambos algoritmos, relacionándolas con el análisis teórico. Explicad si las diferencias en los recuentos de operaciones se reflejan en los tiempos medios de CPU. Incluid las gráficas y, si se ha generado uno, el script utilizado en la entrega e indicad la herramienta de IA empleada, las consultas realizadas y las comprobaciones o correcciones efectuadas.

Material que se debe entregar

La documentación constará de los siguientes apartados:

1. **Introducción:** Descripción técnica del trabajo, de los objetivos y de los datos de entrada y salida del programa.
2. **Código y resultados:** Código de las funciones implementadas en cada apartado, incluidas sus cabeceras, y explicación de las decisiones relevantes de diseño. Tablas de datos, gráficas, comparación con los resultados teóricos y análisis de las diferencias. Indicad los tamaños, el número de permutaciones, las semillas utilizadas, el entorno de ejecución y las opciones de compilación.
3. **Cuestiones:** Respuestas a las cuestiones teóricas, integradas en la memoria, con las consultas a la IA y las verificaciones solicitadas.

La memoria se entregará en formato PDF y su portada incluirá los nombres de los miembros de la pareja. Reunid en un único fichero comprimido, en formato ZIP o TGZ, la memoria, los códigos fuente y las cabeceras, el Makefile, las tablas de datos y los scripts utilizados para generar las gráficas, si los hubiera. El nombre del fichero será **nombre1_apellido1_nombre2_apellido2.zip** o **nombre1_apellido1_nombre2_apellido2.tgz**, donde **nombre_apellido** identifica a cada miembro de la pareja.

Además, cada estudiante deberá conservar una copia de las prácticas en un medio de almacenamiento (memoria USB, CD o DVD, disco duro, almacenamiento remoto, etc.) y tenerla disponible el día del examen de prácticas a final de curso.

Para normalizar la compilación y ejecución de los programas, se sugiere utilizar `make` con el fichero `Makefile` incluido en `practice1.zip`. El `Makefile` ya incluye los objetivos `exercise1`, `exercise2`, ..., `exercise5` para compilar los programas, así como `exercise1_test`, `exercise2_test`, ..., `exercise5_test` para ejecutarlos. Compilad cada programa antes de ejecutar su objetivo de prueba y ajustad los parámetros de ejecución. Para el apartado 6 se reutilizarán los programas de prueba de los apartados 4 y 5. También se pueden emplear entornos de desarrollo integrados.

Instrucciones para la entrega de la memoria y el código de la práctica

El fichero comprimido con la memoria y el código de la práctica se entregará en la fecha correspondiente a vuestro grupo a través de la plataforma Moodle (<https://moodle.uam.es/>).